
Demo script — 4 minutes

A tight run that hits every graded requirement, with the RBAC and compliance story as the spine. Reset first so the world is in a known state:

```
npm run db:reset && npm run dev
```

Keep two browser windows side by side (or two profiles) so you can switch personas without re-typing logins — the login page's demo panel fills the form in one click.

0 · Open (20s) — `/login`

"LoadFlow is a freight brokerage ops platform. A broker is **liable** if it dispatches to a carrier whose insurance has lapsed. So the product's whole job is to make that impossible to do by accident."

Point at the demo-accounts panel. *"Every persona is one click. Watch what changes between them — that's the whole demo."*

1 · The compliance gate (60s) — the money shot

Sign in as `dispatch@meridian.com` (Dispatcher).

- **Load board.** Point at the red alert strip: *Redline Logistics — insurance expired. Cobalt Carriers — authority revoked.* Two loads are held. Search/filter live in the toolbar below (`?q=` , status, carrier, "Blocked only").
 - Open **LF-1044**.
 - Stepper is stuck at step 2. Big red banner: **blocked by compliance — 4 unresolved flags.**
 - Read the flags out: *authority revoked, wrong equipment, unapproved commodity, cargo insurance \$50k against a \$96k declared value.* **"These were raised automatically the moment the carrier was assigned — nobody clicked anything."**
 - Under each flag: *"Overriding requires the `load.override_compliance_flag` permission."* **"I'm a Dispatcher. I can't. And notice the app tells me exactly which permission I'd need — it doesn't just hide the button."**
 - "Confirm rate" is disabled too, with the real reason from the state machine.
-

2 · Permissions are real, and they're enforced in the API (45s)

Switch to `ops@meridian.com` (Ops Lead) — same org, same load, **one extra permission.**

- Same page. Now **Override** is live. Click it → it demands a **written reason** (≥10 chars). Enter one, submit.
- **"Same load, same screen, same org. The only difference is the permission bundle."**

Now the part that matters — drop to a terminal:

```
npm run rbac:proof
```

"This isn't the UI. It signs in as each persona and attacks the REST API directly over HTTP. A Dispatcher hitting the override endpoint by hand gets a 403. A carrier reading a rival's load gets a 404 — not a 403, because a 403 would confirm the load exists. Twenty-four assertions, all green."

Let the green scroll. Land on **24 passed, 0 failed**.

3 · Fix the cause, not the symptom (35s)

Sign in as **admin@redline.com** (the carrier with lapsed insurance).

- **/carrier/compliance** — *"Your insurance lapsed 13 days ago. 1 load is blocked."* Renew the expiry date → save.
- Toast: **"Insurance renewed — 1 load unblocked."**
- Back to the broker board: **LF-1043 is no longer held.** *"Nobody overrode anything. The evaluator re-ran across every live load for that carrier and cleared the flag itself."*

4 · RBAC is a system, not three hardcoded roles (50s)

Sign in as **admin@meridian.com** → **Staff & roles**.

- **"Roles are bundles of permissions, authored here at runtime."** Show the catalog on the right: each capability with the **key the code actually checks** (**load.assign_carrier**) next to it.
- Open **Dispatcher** — 4 permissions. Open **Ops Lead** — the same 4 plus **load.override_compliance_flag** . *"That's the entire difference you just watched play out."*
- Click **+ New role**, tick two permissions, name it. *"Built from the catalog. The code never learns its name."*
- **People** tab: each person's **effective permissions** as chips — the union of their roles.
- **Invitations**: invite a staff member → a copy-able link appears. *"Admins bootstrap by creating the org. Staff can't self-signup — invite only."*

5 · Everything is on the record (30s)

Audit log → hit **"Denied attempts only"**.

- Red rows, each naming the **missing permission key**. *"That's my Dispatcher trying to override, ten seconds after they tried it."* Click a row to expand — the roles they held, the permissions they had.
- **"Denials are audit rows, not console noise. An ops lead can actually review them."**

6 · The other two account types (30s)

- `driver@ironline.com` → a carrier sees **only its own freight** — never the marketplace. Tender accept/decline, status updates, **POD upload** (drag a file) and the inline POD viewer. Note the driver *cannot* accept tenders and the carrier's dispatcher *cannot* update status — disjoint roles inside one carrier.
- `shipper@cascade.com` → read-only, own loads only. **"No roles at all — a shipper's access is pure object-level scoping."** Point at the timeline: *"Milestones only. No rates, no compliance flags, no broker internals. That's a counterparty firewall."*

7 · How I used the AI tool (40s) — required

Show `docs/AI-USAGE.md` and `git log --oneline`, and say three concrete things:

1. **"Next 16 and Prisma 7 are newer than the model's training. So before writing any code, I sent agents into the bundled `node_modules` docs with one rule: don't answer from memory, cite the file."** That caught a project-ender — *Prisma 7 has no built-in SQLite driver; the app literally could not have connected to its own database.*
2. **"I wrote the security-critical spine myself — schema, permission guard, state machine — then froze it, wrote the API contract, and fanned out 13 agents in parallel against it."** Disjoint files, one interface, fit together on the first typecheck.
3. **"Then I turned the model on its own finished work — an adversarial audit, six agents told to *break it, not praise it*."** It found a **live compliance bypass** (override a flag for one carrier, re-tender to another, and the gate let it through) and a **password-hash leak** to shippers — none of which failed a typecheck, a lint, or the test suite, and two of which the app's own docs claimed it prevented. *"The model is a capable builder and an even better critic, but only when you point it at the work and tell it to break it."*

Close on: **"Working must-haves, all three stretch goals, every claim in the README backed by a test or a curl command — and a deployed URL."**

Pre-flight

- ☐ `npm run db:reset` (fresh world; LF-1043 and LF-1044 blocked)
- ☐ `npm run dev` up on :3000
- ☐ A second terminal ready with `npm run rbac:proof` typed but not run
- ☐ An image or PDF on the desktop to drag into the POD uploader
- ☐ Browser zoom ~110%, dark mode on